Rishabhsinh Virpura

Eduardo Gonzalez

Wilson Huang

Bailin Liao

Ebube Jack-Davies

5/02/2025

Professor David Strimple

Senior Design CSE Team 51

RC Car Remodeling Documentation

Introduction

RC cars have been a popular hobby for a long time, they have been a toy for children and adults to play with. They typically involve the user controlling the car with a simple, physical remote control. However, this physical controller results in the technology of the car being rather limited. For example, if a company wanted to add a camera onto the car, the user would need to be able to control the car with the remote and then look at the camera with another device. The remote is also a limitation in the sense that it is necessary to drive the car. The user has to charge it and cannot lose it. Sometimes, this remote-controlled style of controlling an RC car can be limiting.

With modern technology, our group has decided to upgrade RC cars in a few ways to improve the user experience. The main way is by allowing a user to control the car online by using a website. This simplifies the process of controlling the car by taking advantage of the fact that most people have access to their phone or a laptop. By having the user control the car with a website, we can also implement new features such as allowing the user to view through a camera attached to the car. This is a potential feature that could only be done by a web implementation and would be impossible with the simple remote control these cars usually have.

Besides making an RC car more fun and convenient, these kinds of changes can have professional applications as well. Remote controlled drones are currently being designed and used to inspect areas that are too dangerous for humans. This could include inspecting generators in electrical and power plants. Although we are mostly using hobby level technology, a worker can conveniently control a drone like this car by signing into a website to do their job.

This project was split into two main sections: a hardware section and a software section. The hardware section involved upgrading and adding new technology to an existing RC car. We

added many parts to the car, but the most important piece of hardware is likely the raspberry pi. The raspberry pi can connect to the university Wi-Fi, receive instructions from the user on the website, and then control the car itself. It acts as the “brain” of the car. The software section involved actually building the website and server needed to control the car itself. We used a flask backend server which hosted the website itself and communicated with the raspberry pi on the car.

Overall, we believe this technology has potential to be useful as an improvement to already existing RC cars and could be useful on a professional level if it is developed enough.

The key goals for our project are listed below:

1. Develop software to remotely control the car from a website.
2. Add new hardware components to the car such as a raspberry pi, speed controller, camera, and battery.
3. Make sure the connection between the web server, Raspberry Pi, and browser is stable and able to properly control the car.

Hardware

To build this prototype we had to attach new hardware onto an existing RC car. Below is an extensive list of the components we used for this project.

RC Car Chassis: We specifically used the NKOK Off-Road Crawler. This is because the motors of this specific car were compatible with the speed controller we wanted to use, as described in the speed controller section below. We removed the radio communication, existing speed controller, and plastic outer shell. We only used the chassis, wheels, and existing motors of this car as a base for our project.

Speed Controller: For our project we used the l2984N speed controller to control the motors and directly. It is wired to the Raspberry Pi and the motors in the car. The raspberry pi sends the speed controller instructions, which it then uses to control the motors. We chose this speed controller because it worked well with the raspberry pi.

Raspberry Pi: The Raspberry Pi connects to the internet and is used to receive instructions from the server and communicate with the rest of the car.

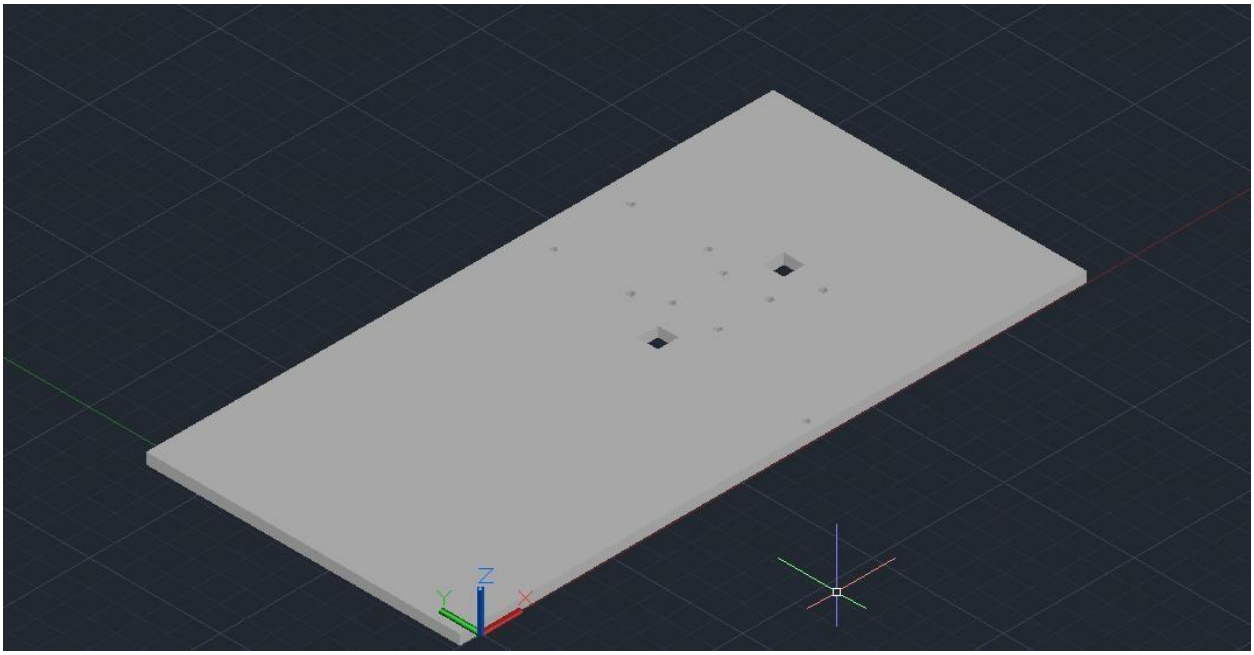
Raspberry Pi Camera: We used a compatible raspberry pi camera to take footage from the car's point of view and relay it to the website. It is directly attached to the raspberry pi itself.

Raspberry Pi Heatsink: Although it was not originally part of our plan, we found the raspberry pi heatsink necessary since the raspberry pi overheated often during testing. After installing the heatsink, the pi began working much more consistently and we did not encounter nay more heating issues.

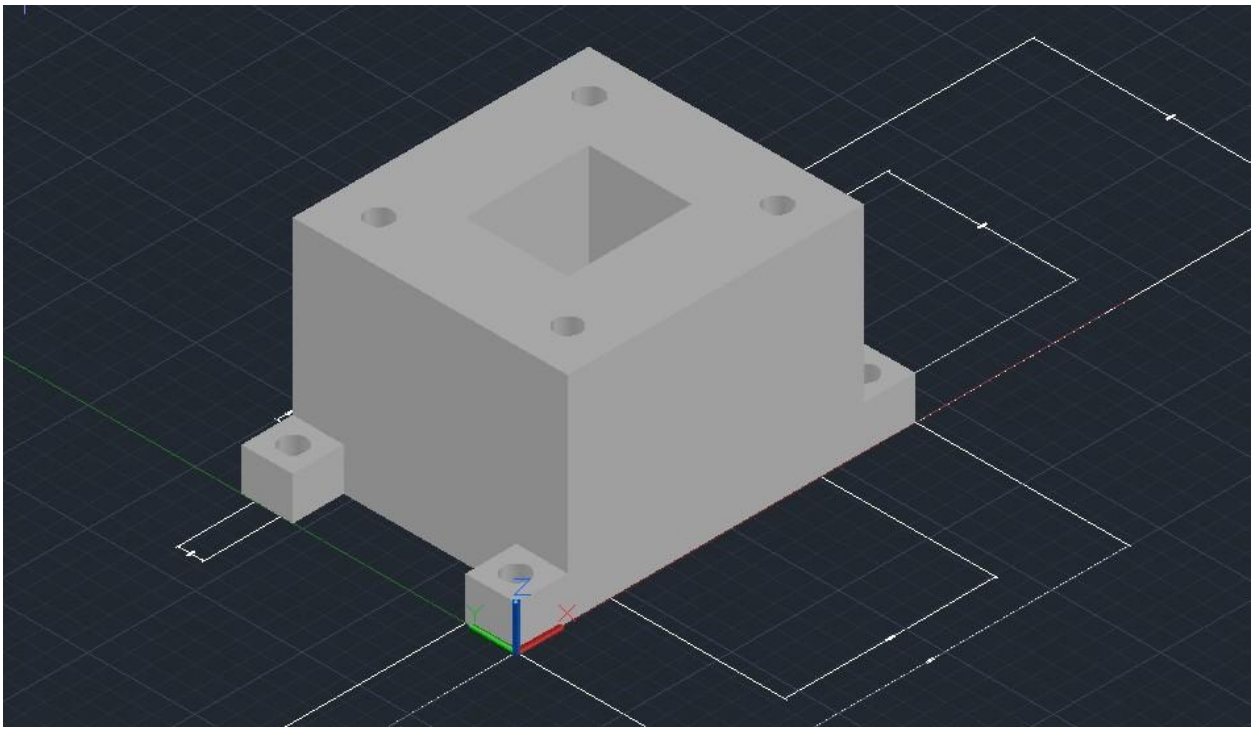
12V Battery: The 12 V battery is a power source used to power the speed controller. We tried using a 6v battery, but it did not provide enough power and made the car quite slow, so we switched to a 12 V one.

Power Bank: The power bank is a power source used to power the raspberry pi. Any generic power bank can work for this purpose, and we simply used one a team member already owned.

3d Printed Parts: We also created and printed 2 3D printed parts. These were made to house all the other components on top of the car with ample space, and to give them a place to screw onto. Below are some pictures of these parts.



This 3D printed part is a sample plate to house the hardware components of the car. It contains holes to easily screw in the Raspberry Pi and Speed Controller.

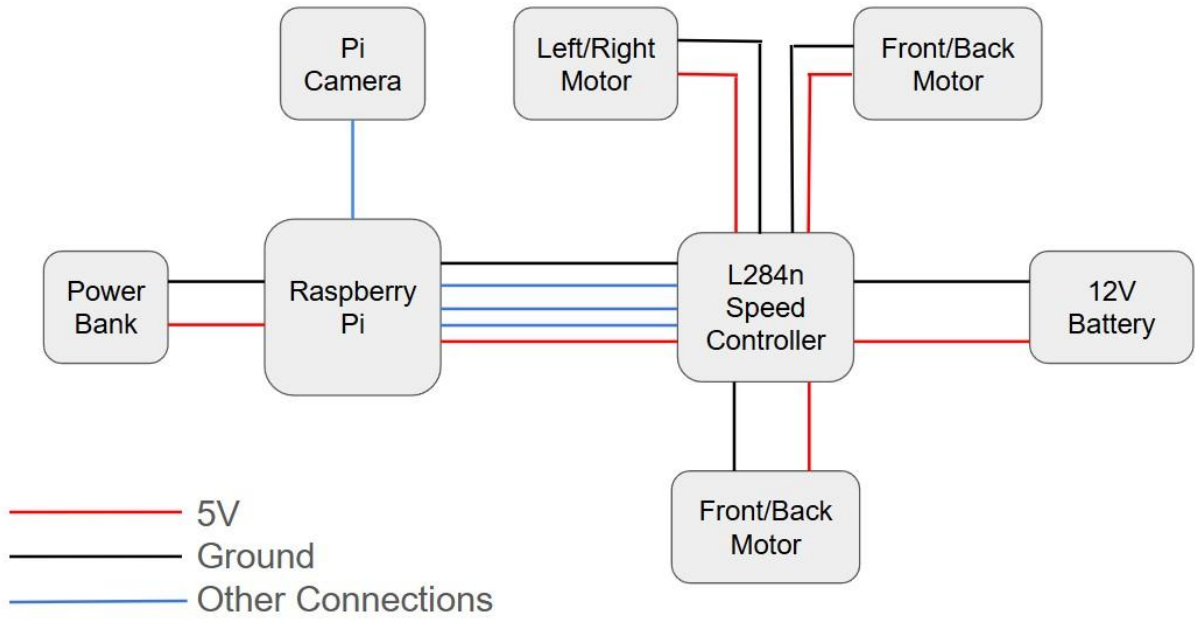


This 3D Printed part is used to screw into the car itself. The top part then screws into this base piece. This is needed because the wheel of the car go a bit above the chassis, so if we attach the top part directly to the chassis of the car they will bump into the wheels. This part elevates the top part a bit to avoid that problem.

Every part listed above was used in order to build the prototype of the car. However, it is important to note that in order to drive the car a server and phone are needed as well. A phone is needed to host the web page that controls the RC car. A server is needed to communicate with the Raspberry Pi after commands are given by the user. The parts listed above are only parts used to build the car itself.

Wiring Diagram

In the end, the hardware components and systems of the car can be represented using this wiring diagram:



Note that the blue lines mean a different type of connection than the red or black lines. For example, the pi camera plugs directly into the motherboard of the raspberry pi, so it is not a 5V or ground connection.

Software

Pi Script

At its core, motor control with Raspberry Pi leverages the General Purpose Input/Output (GPIO) pins to send electrical signals that direct motor movement. However, since the Raspberry Pi can only output 3.3V signals with limited current, direct motor control is impossible for most practical applications. This necessitates the use of motor drivers— specialized circuits that amplify the control signals to power levels suitable for motors.

We first tried using brushless motors because they're efficient and light. But, to be honest, we found it much harder to control these than we thought it would be.

```
from gpiozero import PWMOutputDevice
import time

PIN = 26

motor = PWMOutputDevice(PIN, frequency=50)

def set_motor_speed(duty_cycle):
    motor.value = duty_cycle / 1000
    print(f"Set Motor speed to {duty_cycle}%")
```

We found out that while this approach generated PWM signals, the brushless motor ESC (Electronic Speed Controller) would only respond for brief periods before becoming unresponsive. The oscilloscope showed that our Raspberry Pi was outputting the right PWM signal, so it looks like the problem was how the ESC interpreted these signals over time.

We came up with a more advanced method that includes a proper ESC calibration sequence

```
def initialize(self):
    """Initialize the motor with proper startup sequence"""
    print("\nInitializing ESC...")
    try:
        self.motor = PWMOutputDevice(self.pin, frequency=self.freq, pin_factory=self.pin_factory)

        # Start with neutral position
        print("Setting neutral position...")
        self.motor.value = 0.06
        time.sleep(2)  # Wait for ESC to recognize neutral signal

        # Quick test sequence
        print("Testing forward motion...")
        self.motor.value = 0.075  # Minimum forward
        time.sleep(1)

        print("Testing stop...")
        self.motor.value = 0.06  # Back to neutral
        time.sleep(1)

        print("Testing backward motion...")
        self.motor.value = 0.05  # Minimum backward
        time.sleep(1)

        print("Back to neutral...")
        self.motor.value = 0.06
        time.sleep(1)

        print("ESC initialization complete")
        return True

    except Exception as e:
        print(f"Error initializing ESC: {str(e)}")
        return False
```

Despite careful attention to ESC initialization protocols, we continued to experience intermittent operation. The system would function correctly after rebooting but would become unresponsive after a short period of operation.

The third approach we used was all about the hardware PWM capabilities of the PiGPIO library.

```
def arm(self):
    """Arming sequence with correct signal order."""
    print("\nArming ESC...")
    self.set_throttle(10)  # 2000µs (max)
    time.sleep(2)
    self.set_throttle(5)   # 1000µs (min)
    time.sleep(2)
    print("ESC armed")
```

This implementation gave us more precise timing control, but we still had reliability issues. We thought the ESC's built-in memory and protection systems were making it enter failsafe mode when it got signals that didn't seem right.

After the problems we had with brushless motors, we got new parts, including DC motors and an L298N motor driver. This was a big change for us, moving from high-performance brushless motors to more easily controlled DC motors.

The L298N motor driver represents one of the most common solutions in hobbyist robotics. This H-bridge circuit allows bidirectional control of two DC motors simultaneously, making it perfect for steering and propulsion in wheeled vehicles. The driver requires four GPIO pins to control two motors:

- Two pins for forward/backward movement
- Two pins for left/right steering

This configuration forms the foundation upon which more sophisticated control systems can be built.

Motor control software usually follows a pattern of getting more sophisticated. The simplest way to do this is often with basic directional functions.

The def forward(sec) function is pretty straightforward. You just need to initialize it, then set the GPIO pins to 'True' for 23 and 'False' for 24. Then you just need to sleep for the specified time and clean up the GPIO.

This approach, while functional, suffers from limitations:

- It initializes and cleans up GPIO pins with each function call
- It cannot combine directional controls (e.g., forward + right turn)
- It lacks real-time responsiveness to user input

More advanced implementations address these shortcomings by introducing:

- Persistent GPIO initialization that maintains pin states across function calls
- Parameterized movement functions that accept numerical values for direction •

  Keyboard or joystick interfaces for interactive control

The evolution to keyboard control represents a significant advancement, allowing real-time, intuitive operation:

```
def move(fb_direction, lr_direction, duration=0.1):
    """
    Move the robot with combined forward/backward and left/right control
    fb_direction: 1 for forward, -1 for backward, 0 for stop
    lr_direction: 1 for right, -1 for left, 0 for straight
    duration: how long to run the motors
    """
    # No need to reinitialize GPIO for every move as it causes stuttering
    # Only initialize once at the start of the program

    # Forward/backward control
    if fb_direction == 1:  # Forward
        gpio.output(23, True)
        gpio.output(24, False)
    elif fb_direction == -1:  # Backward
        gpio.output(23, False)
        gpio.output(24, True)
    else:  # Stop forward/backward
        gpio.output(23, False)
        gpio.output(24, False)

    # Left/right control
    if lr_direction == 1:  # Right
        gpio.output(17, False)
        gpio.output(22, True)
    elif lr_direction == -1:  # Left
        gpio.output(17, True)
        gpio.output(22, False)
    else:  # Straight
        gpio.output(17, False)
        gpio.output(22, False)

    if duration > 0:
        time.sleep(duration)
    # Don't cleanup here to maintain continuous control
```

This approach enables smoother operation by maintaining motor states between commands and combining steering with forward/backward movement. However, it still constrains control to the device running the code.

The true potential of Raspberry Pi motor control emerges when combining it with network connectivity. By implementing a client-server architecture using technologies like Socket.IO, control can be extended beyond physical proximity:

- Server-side implementation: A web server handles user connections, authentication, and command queuing
- Client-side interface: A browser-based UI captures user input via keyboard, touch, or custom controls

Pi-side execution: The Raspberry Pi runs client software that connects to the server and executes received commands

This approach offers numerous advantages:

- Remote operation from anywhere with internet access
- Multiple-user support through queuing systems
- Richer control interfaces with visual feedback
- Platform independence (control from any device with a web browser)

This early Socket.IO implementation demonstrates this advanced architecture:

```
@sio.event
def pi_command(data):
    print(f"Received command: {data}")

    # Handle dictionary commands
    if isinstance(data, dict):
        throttle = data.get("throttle")
        turn = data.get("turn")
        throttle_percent = data.get("throttle_percent", 0)
        turn_percent = data.get("turn_percent", 0)

        # Calculate actual speed values (-100 to 100)
        fb_speed = throttle_percent if throttle == "forward" else -throttle_percent if throttle == "backward" else 0
        lr_speed = turn_percent if turn == "right" else -turn_percent if turn == "left" else 0

        # Apply speed control
        set_forward_backward(fb_speed)
        set_left_right(lr_speed)

    # Handle legacy string commands (for backward compatibility)
    elif isinstance(data, str):
        if data == "UP pressed":
            set_forward_backward(100)
        elif data == "DOWN pressed":
            set_forward_backward(-100)
        elif data == "LEFT pressed":
            set_left_right(-100)
        elif data == "RIGHT pressed":
            set_left_right(100)
        elif data == "UP released" or data == "DOWN released":
            set_forward_backward(0)
        elif data == "LEFT released" or data == "RIGHT released":
            set_left_right(0)
```

This event-driven approach decouples the control interface from the execution logic, creating a more modular and extensible system.

Network

Flask-socketIO

Using the Flask-SockletIO library we implemented communication between the users and the Raspberry Pi. We opted to use a client-server architecture to let the users and the car to be connected from any internet source, this also allowed the users to control the car from any distance as far as both clients are connected to the internet. In our project we developed this architecture using socketIO with python on the Raspberry Ri, the SocketIO integration with flask for the SocketIO in JavaScript for the user's browser.

Modules used:

```
from flask import Flask, request, render_template, session, redirect, url_for
from flask_socketio import SocketIO, emit
```

Flask-SocketIO works by having a server program to which SocketIO programs can connect to. The server program takes the interface to which it will be listening to. For example, '0.0.0.0' will listen to all interfaces, which means that it will listen to clients in localhost 127.0.0.1, local network (LAN) 192.168.X.X, and public network (WAN) if available. Our goal was to let users connect seamlessly with their devices therefore we needed to use the public network to connect with the users devices without them having to do any extra steps like changing to the same Wi-Fi the Raspberry Pi was using. Using the public network also allowed the application to let the users control the cat from any distance since the Pi and the user only need access to any type of internet source.

This configuration uses all interfaces and listens to port 4000:

```
354 v if __name__ == '__main__':
355       socketio.run(app, host="0.0.0.0", port=4000, debug=True)
```

Debug=true makes the server program reset every time a change is made

Client-Server Architecture

To connect to the server program by a public network it is necessary to change the router's configuration to allow for outside traffic to access the device hosting the program in the private network. Doing this with the university's internet might have been challenging as we do not have control over the school's routers. To solve that problem we thought of hosting the server program outside the university's network. We used an old laptop to host the server program with my home's internet.

Specifications of the laptop hosting the server:

This allowed us to allow outside traffic from the user's phones into the laptop which forwards the information into the Raspberry Pi. Additionally, this architecture made it possible to have a static server IP and allowed us to have more control over the router's configurations. The Router we used is an Arris nvg448bq which is the modem-router Frontier Internet provides by default. we changed the setting using the configuration IP/URL, in this case http://192.168.254.254/cgi-bin/devices.ha, the URL should be the same for similar routers. The only settings we have to change are the port forwarding options, this will allow for outside traffic to enter my home's network into a specific device and port.

The router's port forwarding configuration

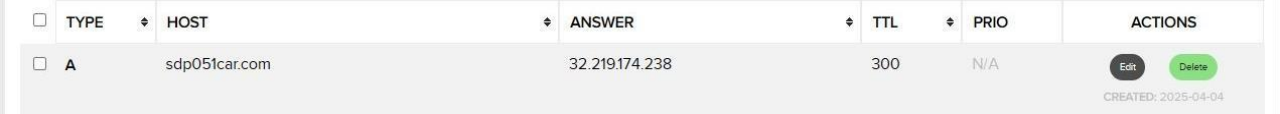
Device	Public IP Address	Service	Ports	Delete
HP	32.219.174.238	HTTP	TCP/UDP: 80	Delete
HP	32.219.174.238	HTTPS	TCP/UDP: 443	Delete
HPWireless	32.219.174.238	flask	TCP/UDP: 5000	Delete
HPWireless	32.219.174.238	samplerver	TCP/UDP: 4000	Delete
HPWireless	32.219.174.238	ssh	TCP/UDP: 22	Delete

Port 22 allowed us to ssh into the laptop to work in the server's programs. Port 4000 and 5000 were used for our socket programs, more specifically, port 4000 was used for the main server program and port 5000 was used for testing or other implementations in case we needed it. The table shows two devices, HP and HPWireless, they are the same device but HP transfers traffic though the ethernet cable while HPWireless uses WIFI. It would be preferable to have all data transferred through cable, however when I changed the port forwarding settings I did not realize there were two different HP devices (the router was using only the private IP address to show devices instead of their name, I assigned friendly names to the private IP addresses later in development), but this should not have much impact as the laptop receives good signal from the modem and the client-server latency was low.

Having opened the proper ports for forwarding and running the program inside my home's network, the program was now ready to receive outside traffic via http://32.219.174.238:4000 . Our goal then was to make this link more user friendly as the URL shows my home's public IP address, the port number we are using, and it uses an http connection, where some browsers might alert the user to avoid connecting to the page because it is not secure.

To remove the public IP address from the URL we purchased a domain in name.com, the following is the A record of the domain, mapping sdp051car.com to my public IP address. The TTL field

represents how long the domain will be cached in the DNS server, for our application this is somewhat irrelevant.



TYPE	HOST	ANSWER	TTL	PRIO	ACTIONS
A	sdp051car.com	32.219.174.238	300	N/A	Edit Delete CREATED: 2025-04-04

Removing the port number was a little more complicated. Browsers use by default port numbers 80 for http and 443 for https, one quick fix would be to host the server into port 80/443 however, those ports are not usable with python programs using Ubuntu, therefore we opted for using an Nginx reverse proxy sever which would forward users from http://sdp051car.com (which uses port 80) to http://sdp051car.com:4000.

The initial Nginx configuration file:

```
server {
    listen 80;
    listen [::]:80;

    server_name sdp051car.com;

    location / {
        proxy_pass http://localhost:4000;
        include proxy_params;
    }
}
```

The firewall settings:

```
}sdp051@HP:~$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
80/tcp                     ALLOW       Anywhere
Nginx HTTP                 ALLOW       Anywhere
Nginx Full                 ALLOW       Anywhere
8081                       ALLOW       Anywhere
8081/tcp                   ALLOW       Anywhere
8082/tcp                   ALLOW       Anywhere
22/tcp                     ALLOW       Anywhere
80/tcp (v6)                ALLOW       Anywhere (v6)
Nginx HTTP (v6)            ALLOW       Anywhere (v6)
Nginx Full (v6)            ALLOW       Anywhere (v6)
8081 (v6)                  ALLOW       Anywhere (v6)
8081/tcp (v6)              ALLOW       Anywhere (v6)
8082/tcp (v6)              ALLOW       Anywhere (v6)
22/tcp (v6)                ALLOW       Anywhere (v6)
```

Using the command *sudo ufw allow 'Nginx HTTP'* and *sudo ufw allow 'Nginx Full'* gives full permissions to Nginx over the http port numbers. The Initial Nginx configuration shows a simple set up to forward clients from port 80 to port 4000. using *sudo certbot --nginx -d sdp051car.com* we were able to get a ssl certificate for the webpage which allowed it to use https instead of http. Finally, the website is accessible by a more user-friendly link that looks like this https://sdp051car.com/ .

Any further improvements would involve the whois protection, we chose not to use it because this was a small project that would be available to the public one day only, but for scaling the project, having the webpage allowing anyone see my personal information is somewhat concerning.

Output for the command whois sdp051car.com

```
Registrant Name: Eduardo Gonzalez
Registrant Organization:
Registrant Street: 13 Legion Dr
Registrant City: EAST HARTFORD
Registrant State/Province: CT
Registrant Postal Code: 06118
Registrant Country: US
Registrant Phone: Redacted for Privacy
Registrant Email: https://www.name.com/contact-domain-whois/sdp051car.com/registrant
Registry Admin ID: Not Available From Registry
Admin Name: Eduardo Gonzalez
Admin Organization:
Admin Street: 13 Legion Dr
Admin City: EAST HARTFORD
Admin State/Province: CT
Admin Postal Code: 06118
```

Inter-Device Communication

For Communication with all devices, we used Socket-IO which is an event driven module that uses the TCP protocol, this makes it easy to use for beginners and allows for faster development times with simpler programs.

Connecting users

RaspberryPi python:

```python
@sio.event
def connect():
    print("Connected to Flask server")
    sio.emit("identify", {"user_agent": "Pi"})
```

Browser JavaScript:

```javascript
socket.on("connect", () => {
    console.log("Connected to server");
    socket.emit("userRequestAdd", { message: "ack" });
});
```

Server python:

```python
@socketio.on("connect")
def handle_connect():
    """Handle new client connections"""
    print(f"Client connected: {request.sid}")

@socketio.on("userRequestAdd")
def handle_user_request_add(data):
    """Handle request to add user to the control queue"""
    user = user_queue.add_user(request.sid)
    print(f"User {request.sid} added to queue")

@socketio.on("identify")
def handle_identify(data):
    """Identify special clients (Raspberry Pi)"""
    global pi_sid
    if data.get("user_agent") == "Pi":
        pi_sid = request.sid
        print(f"Pi connected: {pi_sid}")
        notify_pi_status()
        notify_admins("Raspberry Pi connected to server")
```

This is an example that shows the event driven programming from SocketIO, when a client connects both the server and the client use the connect event, since both the Pi and the browser will use the same event in the server, we made an events to identify them and have different functionalities for both of the clients. For example, when the Pi connects, it will call the server's *connect* event which will only print the session id of the Pi, it will also call its own *connect* event which will call the *identify* event on the server to save the session id as the current RaspberryPi's session id. Similarly, when a user connects with their phone's browser, it will call the server's *connect* event which will only print their session id, and it will call its own *connect* event which then calls the *userRequestAdd* event in the server to be added to the queue to control the car.

Having multiple users

One of our goals early in development was to have many people try out the car during Demo Day, this would mean that multiple users might connect at the same time, therefore we needed to implement a way to let multiple users connect but allow only one user to control the car, otherwise, having multiple people being able to control the car might be confusing and having manual control over who has access to the car complicated. That way we came up with a queue for users to be able to control the car first come first serve.

The structure is a simple circular queue that keeps track of the users with the session id the server saves when they first connect.

```python
class UserQueue:
    def __init__(self):
        self.queue = []  # Each element has 'sid', 'timeAllowed', and 'timeRemaining'
        self.current_index = None  # Index of the user with control
        self.default_time = 90  # Default time allowance in seconds

    def get_current_user(self):
        """Get the user currently in control. {'timeAllowed' = , 'sid' = }"""
        if self.current_index is None or len(self.queue) == 0:
            return None
        return self.queue[self.current_index]

    def activate_current_user(self):
        """Send activation signal to the current user"""
        if self.current_index is None or len(self.queue) == 0:
            return

        current_user = self.get_current_user()
        print(f"Activating user: {current_user['sid']}, time allowed: {current_user['timeAllowed']}s")
        emit('timestart', current_user['timeAllowed'], to=current_user['sid'])

    def next_user(self):
        """Move to the next user in the queue"""
        if len(self.queue) == 0:
            self.current_index = None
            return

        if len(self.queue) == 1:
            # only one user
            self.current_index = 0
        else:
            self.current_index += 1

        if self.current_index >= len(self.queue):
            # reset index position
            self.current_index = 0

        self.activate_current_user()
        notify_admins_queue()

    def add_user(self, session_id, time_allowed=None):
        """Add a user to the queue"""
        if time_allowed is None:
            time_allowed = self.default_time

        user = {
            "sid": session_id,
            "timeAllowed": time_allowed,
            "timeRemaining": time_allowed
        }
        self.queue.append(user)
```

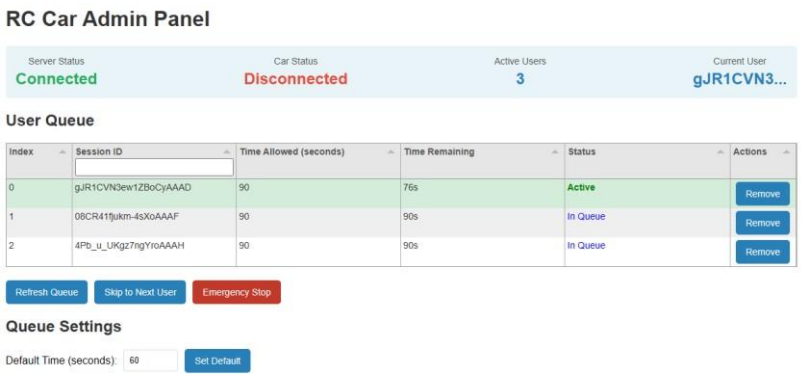
Timing Users

Browsers keep track of how much time has passed, and when the time passed goes over the time granted variable it calls the *timeover* event to let the server skip to the next user.

The Users browsers are told when their time starts by the serber using the *timestart* event, in the message sent by the server there is also how much time that user can control the car for.

Manual Control over the queue

In case we needed to manually change to the next user, or in case there where any errors at the time of presenting the car, we developed an admin page accessible with https://sdp051car.com/login which redirected us to https://sdp051car.com/admin after inputting the correct user and password just in case any users noticed the page.



When a user connects to the admin page it requests the current user in control and the queue which has to be made out of a dictionary of the form {sessionid: value, timeallowed:value}.

The page uses the tabulator module to make the table and socketIO to add functionality.

```
// Queue updates
socket.on('adminResponseQueue', (data) => {
    // Update queue data
    queueData = data;
    console.log(data);
    // Convert the data for Tabulator
    const tableData = data.queue.map((user, index) => {
        return {
            ...user,
            position: index
        };
    });

    // Update table
    table.replaceData(tableData);

    // Update status info
    activeUsersEl.textContent = data.queue.length;

    const currentUserIndex = data.current_index;
    if (currentUserIndex !== undefined && currentUserIndex !== null && data.queue.length > 0) {
        const shortened = data.queue[currentUserIndex].sid.substring(0, 8) + '...';
        currentUserEl.textContent = shortened;
    } else {
        currentUserEl.textContent = 'None';
    }

    addLogEntry(`Queue updated: ${data.queue.length} users`);
});
```

Communicating to the Pi

To allow the users to communicate with the Pi can control the car, we use JavaScript to measure the input from a scale from –100% to 100%, the browser sends only multiples of 5 only when they are not the same value. We decided to take this approach because having too many packets sent using socketIO might cause delays because it uses the TCP protocol. This way we can reduce inputs like [0, 2, 3, 2, 6, 7, 23, 32, 74, 100] to [0, 5, 20, 30, 70, 100]. This was especially important considering the sendControlCommand() function would be called every time a user moved their finger and it is physically hard to keep a finger completely still, as any sort of little movement would trigger the function

```
// Function to send control commands
function sendControlCommand(throttleVal, steeringVal) {
    if (!hasControl) return;

    let roundedT = Math.round(throttleVal / 5) * 5
    let roundedS = Math.round(steeringVal / 5) * 5

    if ( (roundedT == prevT) && (roundedS == prevS)) return;

    prevT = roundedT;
    prevS = roundedS;

    // Send command to server with percentage values
    socket.emit('controlData', {t: roundedT, s: roundedS});
}
```

The server code is simpler as it only forwards the calculated data to the Pi.

```python
@socketio.on("controlData")
def handle_message(data):
    """Handle control messages from users"""
    global pi_sid

    if not pi_sid:
        print("Received control command but no Pi is connected")
        print("INFO: ", data)
        return

    # Only process commands from the current active user
    current_user = user_queue.get_current_user()
    if current_user and current_user['sid'] == request.sid:
        # Forward the command to the Pi
        emit('pi_command', data, to=pi_sid)
        print(f"Forwarded command to Pi: {data}")
```

And Finally, the Raspberry Pi receives the data and calls the functions to use the received values properly.

```python
@sio.event
def pi_command(data):
    print(f"Received command: {data}")

    throttle_percent = data['t']
    turn_percent = data['s']

    # Apply speed control
    set_forward_backward(throttle_percent)
    set_left_right(turn_percent)
```

Webpage and UI

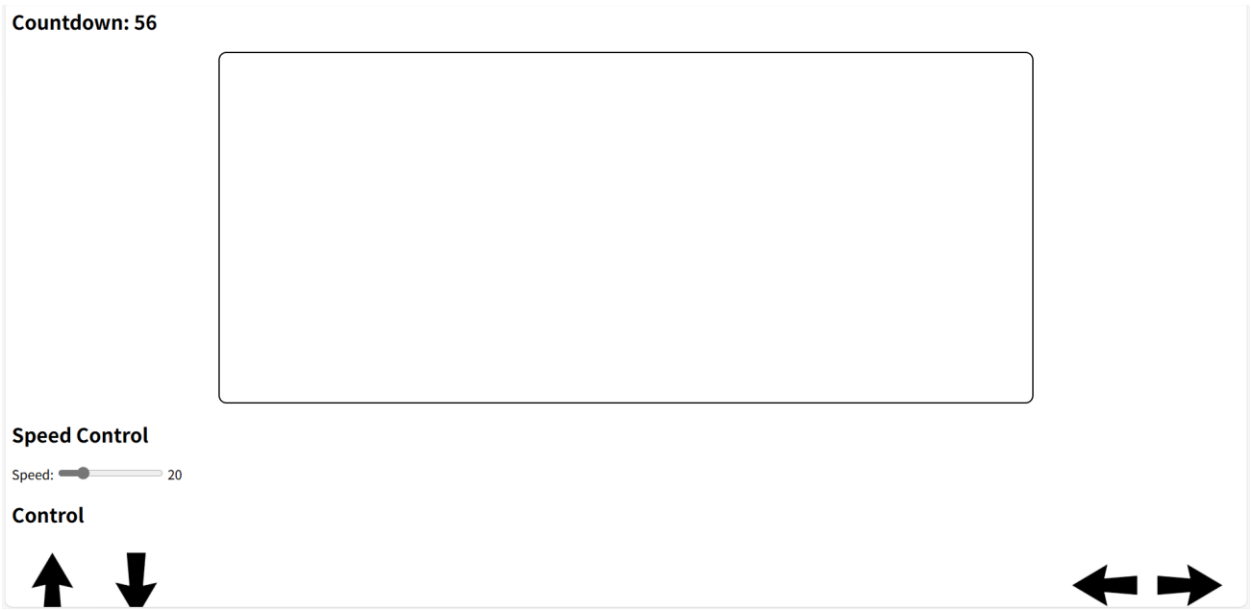
One of the goals of our group's project is to allow users to control a model car using a website. The overall layout of the webpage uses HTML code, which can be divided into several parts according to the functions we need. The first is a timer, which is used to limit the available time of each user. For example, the time is set to 60 seconds. During these 60 seconds, this user can control the model car, and only this user can control it. After the countdown ends, this user will enter a waiting period, and control will be transferred to the next user.

```
<h2 id="count">Countdown: <span id="countdown">60</span></h2>

<script>
    let timeLeft = 60;

    function updateCountdown() {
        if (timeLeft > 0) {
            document.getElementById("countdown").textContent =
timeLeft;   timeLeft--;
        } else {
            document.getElementById("countdown").textContent = "0";
            clearInterval(timer);
        }
    }

    let timer = setInterval(updateCountdown, 1000);
</script>
```

The next function is relatively simple, just a screen that can carry the camera image. The layout code is very simple. The key is how to connect and call the camera image to the website, which will be explained later.

The last and most important part of the website is the function of controlling the RC model car. We imagine using a slider to control the speed of the car, and then there are four buttons below the slider to control the car's forward, backward, left and right turns.

So, this is what our first version of the website look like in the end:



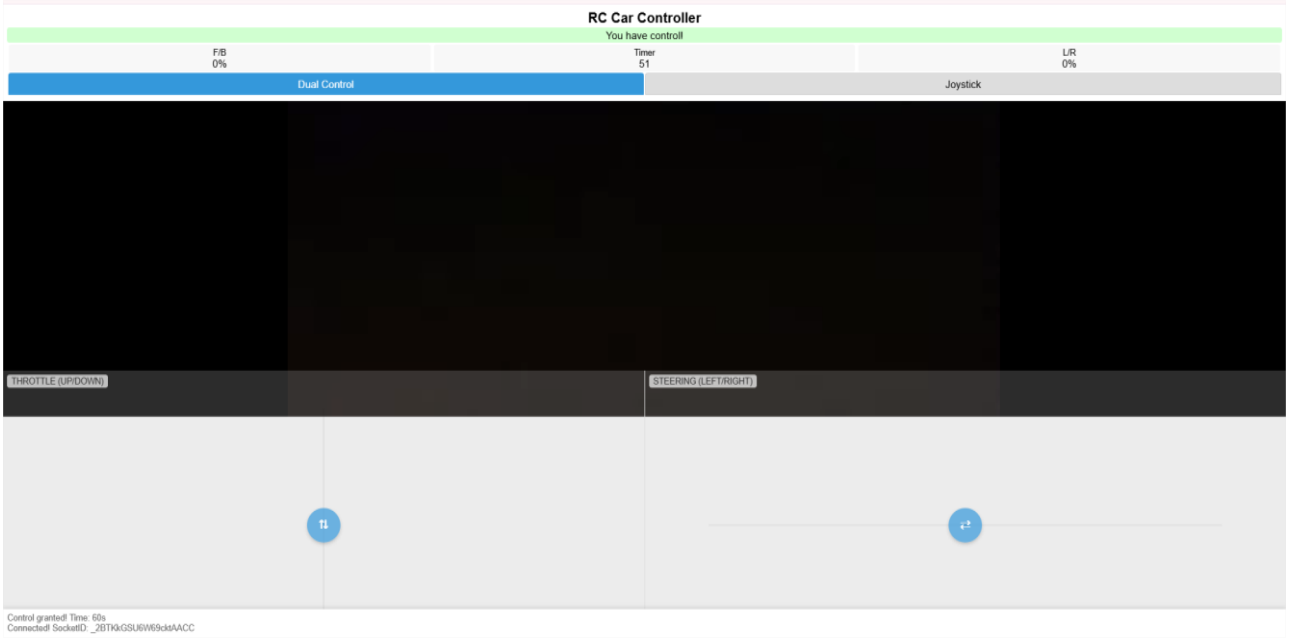
Of course, we feel that this version of the website is too simple and concise, and the control of sliders and buttons will appear cumbersome. So, we improved the website, combining sliders and buttons for more convenient control, and filling the website as much as possible to make it look less simply. We also provide two control methods, one is to control with two sliders, the left one controls the forward/backward of the car, and the right one controls the left/right turn of the car. The second method is to control with a single joystick. At the same time, we set the origin for both methods and provide speed or angle according to the distance of the current slider from the far point. The size of the speed and angle will be displayed above.

The following code is the code for two control methods.

```
    <div class="controls">
        <!-- Bars Control Method -->
        <div id="barsMethod" class="control-method active">
            <div class="bars-control">
                <div id="throttleArea" class="throttle-area" style="top: 40%;">
                    <div class="center-marker dot-marker" style="left: 50%; top: 50%"></div>
                    <div class="guide-line vertical-line"></div>
                    <div id="throttleIndicator" class="control-indicator throttle-indicator" style="top:
50%;">⇅</div>       <div class="control-label">THROTTLE (UP/DOWN)</div>
                </div>

                <div id="steeringArea" class="steering-area" style="top: 40%;">
                    <div class="center-marker dot-marker" style="left: 50%; top: 50%"></div>
                    <div class="guide-line horizontal-line" style="top: 50%;"></div>
                    <div id="steeringIndicator" class="control-indicator steering-indicator" style="top:
50%;">⇄</div>       <div class="control-label-2" style="margin-right: auto;">STEERING (LEFT/RIGHT)</div>
                </div>
            </div>
        </div>

        <!-- Joystick Control Method -->
        <div id="joystickMethod" class="control-method">
            <div class="joystick-control">
                <div id="joystick" class="joystick">
                    <div class="center-marker dot-marker" style="left: 50%; top: 50%"></div>
                    <div class="guide-line joystick-cross-v"></div>
                    <div class="guide-line joystick-cross-h"></div>
                    <div id="joystickIndicator" class="control-indicator joystick-indicator">+</div>
                </div>
            </div>
        </div>
    </div>
```

The following code is for the timer, the speed/angle numbers display, and the screen that hosts the camera video.

```
<div class="header">
        <h1>RC Car Controller</h1>
        <div id="statusDisplay" class="status">Connecting...</div>
        <div class="values">
            <div class="value">
                <small>F/B</small>
                <div id="throttleValue">0%</div>
            </div>
            <div class="value">
                <small>Timer</small>
                <div id="countdown">wait</div>
            </div>
            <div class="value">
                <small>L/R</small>
                <div id="steeringValue">0%</div>
            </div>
        </div>
        <div class="control-tabs">
            <div class="tab active" data-method="bars">Dual Control</div>
            <div class="tab" data-method="joystick">Joystick</div>
        </div>
    </div>
    <div style="overflow: hidden; max-width:100%;height:auto;">
        <iframe
      src="https://passing-repeated-forwarding-custom.trycloudflare.com/"
      width="100%" height="480" style="border:none; display: block; margin: 0 auto; object-fit:contain;" >
    </iframe>
    </div>
```

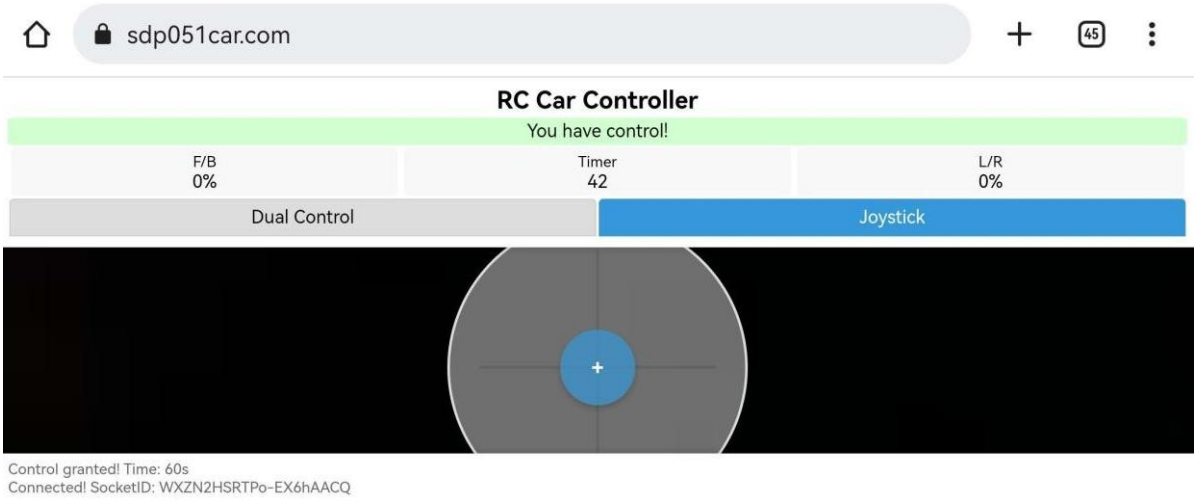


The above picture is the final display of our website. From top to bottom, there is the title, followed by the control prompt. When the user has control, it will show green and "You are controlling" prompt, and if not, it will show red and "Waiting" prompt. Below the prompt is divided into three parts. The left side shows the speed of the car forward/backward, the middle is the countdown, and the right side shows the angle of the car turning left/right. In the middle is the screen used to carry the camera video, and below the screen are two modules for controlling the car.

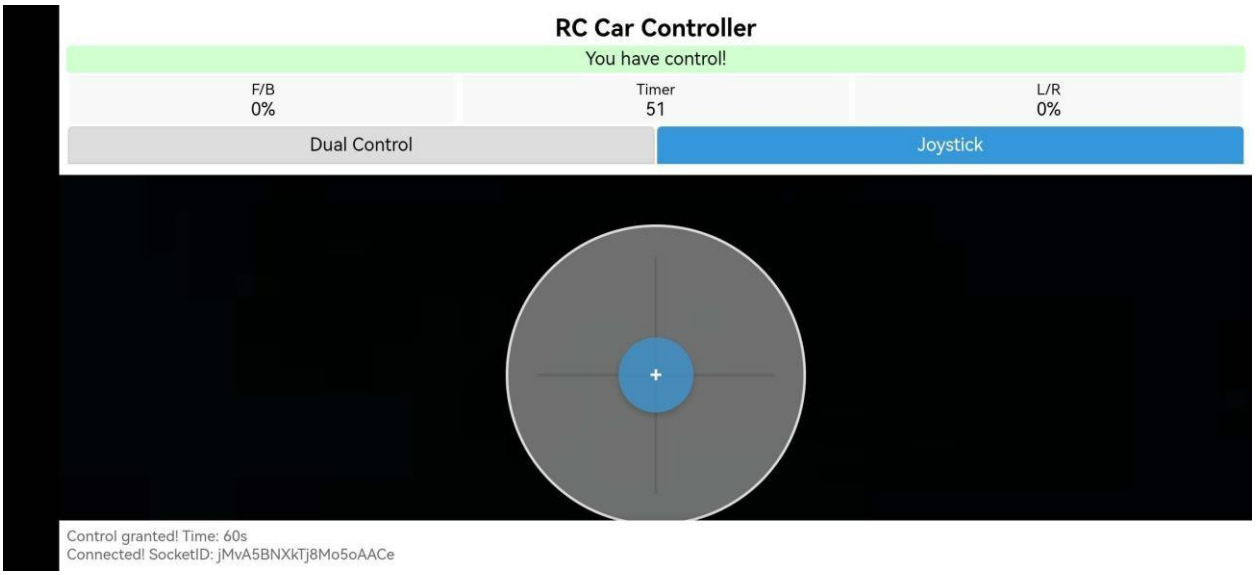
Comparing the two versions of the website layout, it can be clearly seen that the second version is more compact and complete, while the control method is also simpler and there are more prompts to help users control.

Our idea is that the website can be used on both computers and mobile phones, but because the screen width of mobile phones is different from that of computers, we encountered some problems when we first tested the website on mobile phones. For example, when the website is displayed in landscape mode on a mobile phone, not all functions of the website can be displayed, resulting in half of the controller

being off-screen. As shown in the figure:



Our solution to this is that when it is detected that the website is displayed in landscape mode on a mobile phone, if the user touches the screen, it will enter full screen display, as shown in the figure:



Here is the corresponding code:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-
scalable=no">
<style>
    @media screen and (orientation: landscape)and (max-width: 1024px) {
        .joystick{
            top: 50%;
        }
    }
    @media screen and (orientation: landscape) {
        .joystick{
            top: 125%;
        }
    }
    @media screen and (orientation: portrait) and (max-width: 1024px){
        .joystick{
            top: 125%;
        }
    }
</style>

<script>
    function tryFullscreen() {
      const elem = document.documentElement;
      if (elem.requestFullscreen) {
        elem.requestFullscreen();
      } else if (elem.webkitRequestFullscreen) {
        elem.webkitRequestFullscreen();
      } else if (elem.msRequestFullscreen) {
        elem.msRequestFullscreen();
      }
    }

    function setupFullscreenOnLandscape() {
      window.addEventListener("click", () => {
        if (window.matchMedia("(orientation: landscape) and (max-width: 1024px)").matches) {
          tryFullscreen();
          const notice = document.getElementById("notice");
          if (notice) notice.style.display = "none";
        }
      });
    }

    window.onload = setupFullscreenOnLandscape;
</script>
```

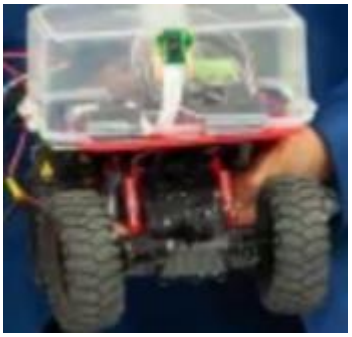
The style shows the positions of the remote-control stick under three screens: computer screen, mobile phone vertical screen and mobile phone horizontal screen. The script detects whether the mobile phone is in horizontal screen when the user touches the screen, and if so, enters full screen mode.

To make it easier for users to use the web page, we have a QR code that can be scanned to directly enter the web page:

Pi camera implementation

For the camera, our goal was to allow users to not only control the car remotely via a web interface but also view the environment through the perspective of the car itself. This kind of feature is typically impossible with standard RC remotes but becomes feasible when the car is controlled via the internet and powered by a micro-controller like the Raspberry Pi.

To accomplish this, we used the official Raspberry Pi Camera Module v2, which plugs directly into the CSI camera port on the Raspberry Pi. The camera was physically mounted on the car and aimed forward to simulate a driver's viewpoint.



Our earliest attempt at video streaming used the picamera2 library along with Python's built-in HTTP server libraries. This setup generated an MJPEG stream (essentially a series of JPEG images) and served it on a local network over HTTP. This solution worked reasonably well for quick prototyping and allowed us to verify the camera was functioning properly. However, this method came with two major limitations:

- No HTTPS Support: Modern browsers block or heavily restrict HTTP content unless it is served over HTTPS. Since our goal was to serve the camera stream directly on the website, this made the MJPEG stream unusable for our end users unless they manually bypassed security warnings.

- Local Only: This solution worked on the same network as the Raspberry Pi but was inaccessible remotely. Our project demanded internet-based control, and thus required a video feed that could be accessed from anywhere.

Next, we attempted to stream the camera feed to YouTube using RTMP (Real-Time Messaging Protocol). We used libcamera-vid to capture video and piped the output to ffmpeg, which encoded and sent it to YouTube's streaming servers. This setup allowed us to embed a YouTube iframe in our website and display the stream there. While this was functional and easy to integrate on the front end, it had two notable drawbacks:

- High Latency: There was a delay of several seconds between the camera and the live stream on YouTube. This made real-time driving impractical.
- Buffering and Quality Warnings: YouTube's encoding algorithms sometimes misinterpreted the stream due to nonstandard timestamps or inconsistent bitrates. This resulted in viewer buffering, black bars, or error messages such as "YouTube is not receiving enough video to maintain smooth streaming."

We optimized encoding parameters in ffmpeg, added audio streams to stabilize playback, and adjusted resolution and bitrate. These tweaks helped barely, but latency and consistency were still not good enough for a responsive control experience for a user.

Ultimately, the solution that worked best was using Cloudflare Tunnel. This service securely tunnels a local HTTP or HTTPS web server to a public, internet-accessible URL without needing to configure the router, firewall, or port forwarding — a perfect fit given we were operating on a university network with restricted router access.

We hosted the control interface and camera stream using Flask on the Raspberry Pi itself. Then we created a public HTTPS URL that tunneled directly to the Pi's local web server. Cloudflare automatically handled the encryption and certificate management.

The camera feed was embedded directly into the Flask web page using an <img> tag . With the tunnel in place, the page and stream became accessible from any browser on any device, including phones.

This final setup had several important benefits:

- Low Latency: The video feed updated almost in real time, with minimal delay (subsecond), allowing smooth and accurate remote control.
- No Setup Barrier: Users did not have to deal with YouTube or third-party platforms — everything was contained on the same control page.
- Secure Access: Thanks to Cloudflare, all data was served over HTTPS using a randomly generated domain, avoiding browser security warnings.
- Cross-Platform Support: The stream worked smoothly on both mobile and desktop platforms, adjusting automatically to screen sizes and resolutions.

Conclusion

Overall, we were able to get the car to work successfully. On the hardware end, we managed to successfully obtain and connect the parts we needed together. On the software end, we managed to create 3 main applications that worked together to control the car. In terms of overall improvements, we likely should have created an outer shell or better way to organize the wires on the car. A common testing issue was how the wires would often come undone while trying to run the car. A more stable solution for the wires would have been preferable. Another issue was how we received our parts quite late into our project. We received them in our second semester, which did not give us much time to build and implement our systems. However, we still managed to make a version of our system that was stable in the end.